

MARITIME MIDDLEWARE

Product Requirements Document

Telix WT3 · Signal Ocean · Claude AI · Custom Email Client

Field	Detail
Document version	1.1 — revised to match agreed architecture (IMAP MCP only)
Status	Draft — for developer review
Date	July 2026
Owner	Panagiotis Dimitrakopoulos
Classification	Confidential
Target developer	Full-stack (Python + React/Electron)

Revision note (v1.1): removed the Postfix Milter / Exchange Transport Agent / AutoHotkey ingestion path that appeared in v1.0. That was an earlier, mail-server-side fallback design. It is not part of what is being built. Ingestion is IMAP MCP only — Telix WT3 → IMAP mailbox → IMAP MCP server → Claude with MCP tools, exactly as shown in the current architecture diagram. All references below have been updated accordingly.

01 OVERVIEW & PROBLEM STATEMENT

1.1 Problem

A shipping broker receives dozens of inbound cargo order emails per day via Telix WT3. For each order, the broker must manually cross-reference Signal Ocean to identify available vessels that match the cargo type, quantity, load port, discharge port, and laycan. This process takes 10–15 minutes per order and is entirely manual.

During peak periods, this creates a bottleneck where orders are not acted on quickly enough, resulting in missed fixtures.

1.2 Solution

Build a middleware system that reads inbound cargo emails automatically via IMAP, parses them using Claude, queries Signal Ocean for available vessels, and ranks the best matches — displaying results to the broker in real time inside a custom email client.

The broker's core workflow does not change. They open an email. The ranked vessel matches are already there.

1.3 Goals

- ▶ Reduce time-to-match from 10–15 minutes to under 30 seconds per inbound order
- ▶ Achieve parsing accuracy above 95% on core fields (cargo, ports, quantity, laycan)
- ▶ Surface top 5 vessel matches ranked by a weighted scoring algorithm
- ▶ Display results in a custom email client — a functional clone of Telix WT3
- ▶ Maintain zero impact on the broker's existing WT3 workflow during transition

1.4 Non-Goals

- ▶ The system will NOT send emails, confirm fixtures, or communicate with counterparties
- ▶ The system will NOT replace Signal Ocean — it reads from it
- ▶ The system will NOT modify anything inside Telix WT3
- ▶ The system will NOT make autonomous decisions — the broker decides

02 USERS & USE CASES

2.1 Primary User

Attribute	Detail
Role	Shipping broker
Technical level	Non-technical — comfortable with email clients, not code
Daily workflow	Receives 20–60 inbound cargo emails per day via Telix WT3
Current pain	Manual cross-referencing of WT3 inbox with Signal Ocean tonnage list
Success metric	Sees ranked vessel matches within seconds of an order arriving
Device	Mac or Windows desktop, single or dual monitor

2.2 Core Use Cases

UC-01 Inbound order received automatically

An inbound cargo email arrives in the broker's mailbox. The IMAP MCP server picks it up, Claude parses it, the backend queries Signal Ocean, and ranked matches are pushed to the email client UI — all before the broker has opened the email.

UC-02 Broker opens email and sees matches

The broker opens the custom email client. The left pane shows inbound orders newest first. Clicking any order shows the parsed fields and top 5 ranked vessel matches on the right. Low-confidence fields are flagged amber.

UC-03 Broker identifies a match and acts

The broker reviews the ranked vessels, selects the best match, and proceeds to negotiate in Telix WT3 or Signal Ocean as they would normally. The middleware does not participate in the negotiation or closing.

UC-04 Real-time push of new orders

While the broker is in the email client, a new order arrives. The left pane updates immediately with the new order highlighted. No refresh required.

UC-05 Search historical orders

The broker asks Claude (via the IMAP MCP connection) to search historical orders — for example all grain enquiries from a specific counterparty in the last month.

03 SYSTEM ARCHITECTURE

3.1 High-Level Architecture

The system has five layers. Each layer has a single responsibility and communicates with adjacent layers only via defined interfaces (HTTP, WebSocket, IMAP, stdio). Ingestion is IMAP only — there is no mail-server-side plugin or Windows click-trigger involved.

Layer	Name	Responsibility	Interface In	Interface Out
1	Mail Ingestion	Read inbound emails from the WT3 mailbox via IMAP	IMAP port 993	MCP tool calls to Layer 5
2	Python Backend	Parse emails, match vessels, serve results	HTTP + WebSocket	PostgreSQL reads/writes
3	Database	Persist orders, vessels, matches, history	SQLAlchemy ORM	SQL queries
4	Email Client UI	Display ranked matches to broker	HTTP + WebSocket	Broker interaction
5	Agent System	AI coordination via Claude Code + MCP	MCP stdio	Calls Layers 1 + 2

3.2 Data Flow

Step 1 — Email arrives in the broker's mailbox. The IMAP MCP server reads it and hands the raw body to Claude.

Step 2 — Claude extracts structured fields: cargo type, quantity, load port, discharge port, laycan, vessel type. Each field gets a confidence score 0-1.

Step 3 — Port normaliser maps raw port names to canonical Signal Ocean names using a 400+ entry alias table.

Step 4 — Matching engine scores every vessel in the local Signal Ocean cache against the parsed order across four dimensions. Returns top 5 ranked vessels.

Step 5 — Results are persisted to PostgreSQL and pushed to the email client UI via WebSocket.

Step 6 — Broker sees ranked vessel cards alongside the parsed order. Proceeds normally.

3.3 MCP Architecture

The system uses two MCP (Model Context Protocol) servers — Anthropic's standard for giving Claude access to external tools. Each runs as a separate Python process and communicates with Claude Code via stdio.

MCP Server	File	External System	Tools Exposed
IMAP MCP	middleware/mcp/imap_server.py	WT3 mailbox (IMAP port 993)	get_latest_orders, search_orders, get_order, watch_inbox
Signal Ocean MCP	middleware/mcp/signal_server.py	Signal Ocean API	get_tonnage_list, get_vessel, get_ports, get_distances, get_vessel_classes

Note: MCP servers communicate with Claude Code via stdio — not HTTP. They are configured in `~/.claude/claude_desktop_config.json` and loaded automatically when Claude Code starts. IMAP is the sole ingestion mechanism — there is no separate mail-server plugin (Milter, Exchange Transport Agent) and no Windows AutoHotkey click-trigger in this design.

3.4 Agent System

Five specialised Claude agents are defined as markdown files and coordinated via CLAUDE.md — the project's persistent memory file read automatically by Claude Code at the start of every session.

Agent	Role	Primary Responsibility
Agent 1 — CEO	Coordinator	Reads CLAUDE.md, knows all other agents, tracks project status, assigns tasks, updates memory
Agent 2 — Architect	System Design	Makes architectural decisions, designs interfaces, writes decision records to .claude/decisions/
Agent 3 — Coder	Implementation	Writes all Python and JavaScript code, follows existing patterns, builds MCP servers and UI
Agent 4 — Debug/Test	Quality Assurance	Writes pytest tests, validates parsing accuracy, tests MCP connections, runs smoke tests
Agent 5 — Briefing	Reporting	Produces daily summary of inbox orders, match quality, parse errors, Signal cache health

04 FUNCTIONAL REQUIREMENTS**4.1 Email Ingestion — FR-01 to FR-04**

ID	Requirement	Priority
FR-01	System MUST read inbound emails from the broker's mailbox via IMAP without affecting Telix WT3 operation	Must Have
FR-02	System MUST process new emails within 5 seconds of arrival	Must Have
FR-03	System MUST handle MIME multipart emails and extract plain text body	Must Have
FR-04	System MUST use the IMAP MCP server (watch_inbox) as the sole ingestion mechanism — no mail-server-side plugin or client-side trigger is required	Must Have

4.2 Message Parsing — FR-05 to FR-10

ID	Requirement	Priority
FR-05	System MUST extract cargo type, quantity, load port, discharge port, laycan start, laycan end from every inbound order	Must Have
FR-06	System MUST assign a confidence score (0–1) to every extracted field	Must Have
FR-07	System MUST normalise port names to canonical Signal Ocean names using an alias lookup table	Must Have
FR-08	System MUST flag fields with confidence below 0.75 as low-confidence in the UI	Must Have
FR-09	System MUST handle abbreviated shipping message formats (e.g. '55k grain ant/jpn lc aug 10-20')	Must Have
FR-10	System SHOULD achieve parsing accuracy above 95% on core fields across real broker message samples	Should Have

4.3 Vessel Matching — FR-11 to FR-16

ID	Requirement	Priority
FR-11	System MUST score available vessels across four dimensions: vessel size, geography, date overlap, cargo type	Must Have
FR-12	System MUST return the top 5 ranked vessels per inbound order	Must Have
FR-13	System MUST display a score breakdown per vessel showing each dimension's contribution	Must Have
FR-14	Scoring weights MUST be configurable via environment variable without code changes	Must Have
FR-15	System MUST refresh the Signal Ocean vessel cache every 5 minutes in the background	Must Have
FR-16	System MUST display the Signal cache age in the UI so the broker knows data freshness	Must Have

4.4 Email Client UI — FR-17 to FR-24

ID	Requirement	Priority
FR-17	UI MUST display inbound orders in a scrollable left pane, newest first	Must Have
FR-18	UI MUST display parsed fields and ranked vessel cards when an order is selected	Must Have
FR-19	UI MUST update in real time when a new order arrives — no manual refresh required	Must Have
FR-20	UI MUST highlight new orders visually for 8 seconds after arrival	Must Have
FR-21	UI MUST show low-confidence parsed fields in amber with a warning indicator	Must Have
FR-22	UI MUST show vessel name, class, DWT, open port, open date, and total score per match	Must Have
FR-23	UI MUST show a segmented score breakdown bar per vessel (size, geography, date, cargo)	Should Have
FR-24	UI MUST allow the broker to expand the raw email body for verification	Should Have

4.5 Data Persistence — FR-25 to FR-28

ID	Requirement	Priority
FR-25	System MUST persist every inbound order with all extracted fields to PostgreSQL	Must Have
FR-26	System MUST persist every match result with full score breakdown to PostgreSQL	Must Have
FR-27	System MUST log every Signal Ocean cache refresh with timestamp, vessel count, and duration	Must Have
FR-28	System SHOULD retain full order and match history for future analytics and data product use	Should Have

05 NON-FUNCTIONAL REQUIREMENTS

ID	Category	Requirement
NFR-01	Performance	End-to-end latency from email arrival to UI display MUST be under 5 seconds for 95% of orders
NFR-02	Performance	Vessel matching against local cache MUST complete in under 100ms
NFR-03	Performance	UI MUST render match results in under 50ms after data is received
NFR-04	Reliability	Signal Ocean cache failure MUST NOT prevent the UI from loading — degrade gracefully and show cache age
NFR-05	Reliability	LLM parser failure on a single message MUST NOT crash the backend — log error and continue
NFR-06	Reliability	WebSocket disconnection MUST trigger automatic reconnection within 3 seconds
NFR-07	Security	All traffic MUST remain on localhost — no inbound ports exposed to the network
NFR-08	Security	API keys MUST be stored in .env file — never hardcoded in source code
NFR-09	Security	Email content MUST NOT leave the local machine except via the Anthropic API for parsing
NFR-10	Maintainability	Scoring weights MUST be adjustable via .env without redeployment
NFR-11	Maintainability	All schema changes MUST be managed via Alembic migrations — no manual database edits
NFR-12	Scalability	System MUST support multiple broker inboxes via configuration with no architectural changes

06 LANGUAGES & TECHNOLOGY STACK

6.1 Language Map

Three languages are used. Each owns a clearly defined part of the system and does not overlap with the others.

Language	Version	Where Used	Why This Language
Python	3.11+	All backend — API, parser, matching, MCP servers, database, scheduler	Anthropic and Signal Ocean SDKs are both Python-first. No other language has official support for both.
JavaScript (React + Electron)	Node 18+ / React 18	Email client UI only	React for component-based UI. Electron for native desktop window with always-on-top and system tray.
SQL	PostgreSQL dialect	Database schema and migrations	Generated by SQLAlchemy and Alembic automatically. Developers rarely write raw SQL.

6.2 Python Libraries

Library	Version	Purpose
fastapi	0.111.0	REST API and WebSocket server
uvicorn	0.29.0	ASGI server running FastAPI
anthropic	0.26.0	Claude API client — message parsing
signal-ocean	2.4.0	Signal Ocean Python SDK — vessel data
mcp	Latest	Anthropic MCP server framework — IMAP and Signal MCP servers
sqlalchemy	2.0.30	ORM for all database operations
psycopg2-binary	2.9.9	PostgreSQL driver
alembic	1.13.1	Database schema migrations
apscheduler	3.10.4	Background Signal cache refresh every 5 minutes
pydantic	2.7.1	Data validation for all API requests and responses
pydantic-settings	2.2.1	Load all configuration from .env file
structlog	24.1.0	Structured JSON logging
tenacity	8.3.0	Automatic retry with exponential backoff for API calls
httpx	0.27.0	Async HTTP client for internal calls
python-dateutil	2.9.0	Parse date strings from broker messages
pytz	2024.1	Timezone handling

6.3 JavaScript / Node Libraries

Library	Version	Purpose
electron	30.0.0	Native desktop window wrapper
react	18.2.0	UI component framework
react-dom	18.2.0	React DOM renderer
react-scripts	5.0.1	Build toolchain and dev server
electron-builder	24.13.0	Packages app into Windows (.exe) or Mac (.dmg) installer
concurrently	8.2.0	Run React dev server and Electron simultaneously
wait-on	7.2.0	Wait for React dev server before launching Electron

6.4 Infrastructure

Component	Technology	Version	Notes
Database	PostgreSQL	16	Runs locally. Tables auto-created on first startup.
Development IDE	Cursor	Latest	VS Code fork with AI. Claude Code runs inside terminal.
AI agent runtime	Claude Code CLI	Latest	Reads CLAUDE.md on every session. Manages all five agents.
Version control	Git	Latest	Required by Claude Code.
Package manager (Python)	pip + venv	—	Isolated virtual environment per component.
Package manager (JS)	npm	9+	Node package management.

07 CODE STRUCTURE & FILE MAP

7.1 Repository Layout

Path	Language	Description
CLAUDE.md	Markdown	Project memory. Read by Claude Code automatically at every session start.
.claude/agents/	Markdown	Five agent definitions: ceo.md, architect.md, coder.md, debug.md, briefing.md
.claude/decisions/	Markdown	Architecture decision records written by Agent 2 Architect
middleware/	Python	Complete backend — all server-side logic
middleware_frontend/	JavaScript	Electron + React email client UI

7.2 middleware/ — Python Backend

File	Status	Description
main.py	✓ Built	Entry point. Imports FastAPI app, starts uvicorn server.
config.py	✓ Built	All settings loaded from .env. Single source of truth for all configuration.
requirements.txt	✓ Built	All Python dependencies with pinned versions.
api/app.py	✓ Built	FastAPI server. All endpoints: /internal/ingest, /matches/{id}, /orders/latest, /status, /ws
parser/llm_parser.py	✓ Built	Claude API calls, structured field extraction, confidence scoring, pre-processing.
core/ports.py	✓ Built	400+ port alias table, area proximity matrix, geography scoring function.
signal_client/client.py	✓ Built	Signal Ocean SDK wrapper. refresh() pulls tonnage list. get_available_vessels() reads local cache.
matching/engine.py	✓ Built	Four scoring functions (size, geo, date, cargo) and rank() method. Weights configurable via .env.
scheduler/jobs.py	✓ Built	APScheduler job. Calls Signal refresh every 5 minutes. Runs immediately on startup.
mcp/imap_server.py	⚡ To Build	IMAP MCP server. Exposes: get_latest_orders, search_orders, get_order, watch_inbox.
mcp/signal_server.py	⚡ To Build	Signal Ocean MCP server. Exposes: get_tonnage_list, get_vessel, get_ports, get_distances.
database/models.py	✓ Built	SQLAlchemy models: inbound_orders, cached_vessels, match_results, signal_cache_refreshes.
database/migrations/	✓ Built	Alembic migration files. 0001_initial_schema creates all tables and indexes.

database/seed.py	✓ Built	Inserts 14 vessels, 5 orders, 25 matches for development without live API keys.
database/cli.py	✓ Built	CLI: <code>python -m database.cli [migrate seed status reset rollback]</code>
tests/test_core.py	✓ Built	20 pytest unit tests covering parser, matching engine, and port normalisation.

7.3 middleware_frontend/ — JavaScript UI

File	Status	Description
package.json	✓ Built	Dependencies and npm scripts: start (dev), build, dist (installer).
electron/main.js	✓ Built	Electron main process. 420px frameless window docked to screen right. Always on top.
electron/preload.js	✓ Built	Secure IPC bridge. Exposes <code>electronAPI.minimizeWindow</code> , <code>openExternal</code> , <code>dragWindow</code> .
public/index.html	✓ Built	HTML shell with Content-Security-Policy restricting to localhost only.
src/index.js	✓ Built	React entry point.
src/index.css	✓ Built	Design system. Colour tokens (navy/teal/amber), typography, animations, skeleton loader.
src/App.jsx	✓ Built	Root. WebSocket connection, selected order state, layout orchestration.
src/lib/api.js	✓ Built	HTTP + WebSocket client. <code>getLatestOrders</code> , <code>getMatches</code> , <code>getStatus</code> , <code>subscribeToMatches</code> .
src/components/StatusBar.jsx	✓ Built	Top bar. Green/red connection dot, vessel count pill, cache age pill, minimise button.
src/components/OrderList.jsx	✓ Built	Left pane. 40 most recent orders. Polls every 20s. Amber dot on new arrivals.
src/components/OrderCard.jsx	✓ Built	Single order row. Cargo type, quantity, route, time ago, top match score chip.
src/components/MatchPanel.jsx	✓ Built	Right pane. <code>ParsedOrderSummary</code> + <code>DealCard</code> list. Skeleton loading state.
src/components/ParsedOrderSummary.jsx	✓ Built	Two-column field grid. Amber ⚠ on low-confidence fields. Expandable raw body.
src/components/DealCard.jsx	✓ Built	Ranked vessel card. Name, class, DWT, ports, open date, score number, <code>ScoreBar</code> .
src/components/ScoreBar.jsx	✓ Built	Four-segment bar. Blue=size, Green=geo, Amber=date, Purple=cargo. Opacity by score.

7.4 Runtime Processes

Four separate processes run simultaneously. They communicate only via HTTP, WebSocket, IMAP, or stdio — they never import each other's code directly.

Process	Start Command	Language	Port / Protocol	Communicates With
Python backend	uvicorn main:app --port 5000	Python	:5000 HTTP + WS	PostgreSQL :5432, Anthropic API, Signal Ocean API
IMAP MCP server	python mcp/imap_server.py	Python	stdio	Mailbox :993 IMAP, Claude Code
Signal MCP server	python mcp/signal_server.py	Python	stdio	Signal Ocean API, Claude Code
Electron UI	npm start	JavaScript	:3000 (dev)	Python backend :5000 HTTP + WS

08 DATABASE SCHEMA

8.1 Tables

Table	Purpose	Key Fields
inbound_orders	Every parsed inbound email	id (UUID), received_at, sender, subject, raw_body, cargo_type, quantity_mt, load_port_canonical, discharge_port_canonical, laycan_start, laycan_end, parse_confidence, confidence_scores (JSON), has_low_confidence_fields
cached_vessels	Signal Ocean tonnage list cache	id (UUID), vessel_id, vessel_name, vessel_class, dwt, open_port, open_port_area, open_date, commercial_status, market_deployment, latitude, longitude, cargo_types (JSON), cached_at
match_results	Ranked vessels per order	id (UUID), order_id (FK), vessel_id, vessel_name, vessel_class, dwt, open_port, open_date, rank, total_score, score_vessel_size, score_geography, score_date_overlap, score_cargo_type
signal_cache_refreshes	Signal cache refresh audit log	id (UUID), refreshed_at, vessel_count, success, error_message, duration_ms

8.2 Migration Management

- ▶ All schema changes MUST be made via Alembic migration files — never by editing the database directly
- ▶ To create a new migration: `alembic revision --autogenerate -m 'description'`
- ▶ To apply pending migrations: `python -m database.cli migrate`
- ▶ To roll back last migration: `python -m database.cli rollback`
- ▶ Migration files live in `middleware/database/migrations/versions/`

09 MATCHING & SCORING ALGORITHM

9.1 Scoring Formula

Total Score = (0.35 × Size) + (0.30 × Geography) + (0.20 × Date) + (0.15 × Cargo)

Note: *Weights are configurable via environment variables: WEIGHT_VESSEL_SIZE, WEIGHT_GEOGRAPHY, WEIGHT_DATE_OVERLAP, WEIGHT_CARGO_TYPE. Must sum to 1.0.*

9.2 Dimension Scoring

Dimension	Weight	Scoring Logic	Score Range
Vessel Size	35%	Ratio of vessel DWT to cargo quantity MT. Ideal ratio 1.05–1.35 (vessel slightly larger than cargo to account for ballast and stores). Score degrades linearly outside this range.	0.0 – 1.0
Geography	30%	Area proximity between vessel open port area and load port area. Uses a preconfigured proximity matrix (e.g. ARA→ARA = 1.0, ARA→Far East = 0.2). Unknown ports return 0.5 (neutral).	0.0 – 1.0
Date Overlap	20%	How well vessel open date fits laycan window. Best = opens 0–7 days before laycan start. Inside laycan = $0.85 \times (1 - \text{days_in/window})$. After laycan end = 0.0.	0.0 – 1.0
Cargo Type	15%	Compatibility between cargo type and vessel class. Hardcoded matrix: grain→Panamax = 1.0, crude→VLCC = 1.0, grain→VLCC = 0.0. Unknown types return 0.5.	0.0 – 1.0

10 EXTERNAL DEPENDENCIES & API REQUIREMENTS**10.1 API Keys Required**

Service	Source	Used For	Call Frequency
Anthropic API key	console.anthropic.com	Claude message parsing (claude-sonnet-4-6)	Once per inbound email
Signal Ocean API key	apis.signalocean.com	Tonnage List, Distances, Scraped Data APIs	Every 5 minutes (cache refresh)

Note: Signal Ocean API access is separate from the platform subscription. Confirm with your Signal Ocean account manager which APIs are included in your subscription before development begins.

10.2 Signal Ocean APIs Needed

API	Priority	Purpose
Tonnage List API	Essential — cannot build without	Query available vessels by class, area, laycan, commercial status, AIS recency
Distances API	Essential — improves scoring accuracy	Real sea distances between ports and vessel positions for geographic scoring
Scraped Data API	High value — confirm if included in subscription	Structured cargo orders and fixture data parsed from market emails

10.3 IMAP Credentials Required

Field	Description
IMAP host	Mail server hostname (e.g. mail.brokerfirm.com)
IMAP port	993 (TLS) or 143 (STARTTLS)
Username	Broker email address
Password	Email account password or app-specific password

Note: IMAP works the same way regardless of the mail server behind it (Postfix, Exchange, Gmail, etc.) — no server-side integration is required.

11 BUILD PLAN — WHAT IS BUILT VS WHAT REMAINS**11.1 Already Built**

The following components are production-quality and ready to run. A developer can clone the repo, fill in the .env file, and have the system running in under an hour.

Component	File(s)	Notes
FastAPI backend + WebSocket	api/app.py	All endpoints live. Handles concurrent requests.
LLM message parser	parser/llm_parser.py	Claude integration with confidence scoring and retry logic.
Port normaliser	core/ports.py	400+ aliases. Area proximity matrix for 20+ trading regions.
Signal Ocean cache client	signal_client/client.py	Full refresh + upsert + prune cycle. Read-only cache queries.
Matching engine	matching/engine.py	Four scoring functions + ranker. Configurable weights.
Background scheduler	scheduler/jobs.py	APScheduler. Runs Signal refresh on startup and every 5 min.
PostgreSQL models	database/models.py	Four tables with indexes. Auto-creates on startup.
Alembic migrations	database/migrations/	Versioned schema management.
Seed data	database/seed.py	14 vessels, 5 orders, 25 matches — realistic test data.
Database CLI	database/cli.py	migrate, seed, status, reset, rollback commands.
Electron main + preload	electron/	Frameless always-on-top window. Secure IPC bridge.
React email client UI	src/	7 components. Dark maritime theme. WebSocket live updates.
Claude agent files	.claude/agents/	Five agents defined. CEO coordinates via CLAUDE.md.
Unit tests	tests/test_core.py	20 tests. Parser, engine, ports. Run with pytest tests/-v.

11.2 To Build

Component	Priority	Est. Effort	Description
IMAP MCP Server	Critical	2–3 days	Python MCP server at middleware/mcp/imap_server.py. The system's sole ingestion path. Uses IMAP idle/poll to watch the inbox in real time. Exposes get_latest_orders, search_orders, get_order, watch_inbox.
Signal Ocean MCP Server	Critical	2–3 days	Python MCP server at middleware/mcp/signal_server.py. Wraps Signal Ocean SDK. Exposes get_tonnage_list, get_vessel, get_ports,

			get_distances.
WT3 Clone Email Client (full)	High	3-4 weeks	Full email client extending the existing Electron panel. Reads inbox via IMAP MCP. Handles email threading, reply composition stub, full inbox management.
Signal Ocean mock server	Medium	1 day	Mock Signal responses matching SDK format. Flag in .env switches between mock and live. Unblocks frontend development before API key is available.
Multi-broker configuration	Low	1 week	Support multiple broker inboxes via configuration. Each broker has own IMAP credentials and optional scoring weight overrides.
Analytics data export	Low	2 weeks	Aggregated anonymised order flow API. Feeds future commercial data product for banks and hedge funds.

12 SETUP & ENVIRONMENT**12.1 Prerequisites**

Software	Version	Platform	Download
Python	3.11+	Mac + Windows	python.org/downloads
PostgreSQL	16	Mac + Windows	postgresql.org/download
Node.js	18 LTS+	Mac + Windows	nodejs.org
Git	Latest	Mac + Windows	git-scm.com
Claude Code CLI	Latest	Mac + Windows	<code>npm install -g @anthropic-ai/claude-code</code>
Cursor	Latest	Mac + Windows	cursor.com

12.2 First-Time Setup Sequence

- ▶ 1. Clone repo: `git clone <repo-url> maritime-middleware`
- ▶ 2. Create virtual environment: `python -m venv venv && source venv/bin/activate`
- ▶ 3. Install Python dependencies: `pip install -r requirements.txt`
- ▶ 4. Install JS dependencies: `cd middleware_frontend && npm install`
- ▶ 5. Create database: `psql -U postgres -c 'CREATE DATABASE maritime_middleware'`
- ▶ 6. Copy environment file: `cp .env.example .env` and fill in API keys
- ▶ 7. Run migrations: `python -m database.cli migrate`
- ▶ 8. Seed test data: `python -m database.cli seed`
- ▶ 9. Start backend: `uvicorn main:app --host 127.0.0.1 --port 5000`
- ▶ 10. Start UI: `cd middleware_frontend && npm start`
- ▶ 11. Verify: open `http://127.0.0.1:5000/status` in browser — should show healthy

12.3 Environment Variables (.env)

Variable	Required	Description
DATABASE_URL	Yes	<code>postgresql://middleware:password@localhost:5432/maritime_middleware</code>
ANTHROPIC_API_KEY	Yes	Claude API key from console.anthropic.com
SIGNAL_OCEAN_API_KEY	Yes	Signal Ocean API key from apis.signalocean.com
IMAP_HOST	Yes (for MCP)	Mail server hostname
IMAP_PORT	Yes (for MCP)	993 (TLS) or 143
IMAP_USER	Yes (for MCP)	Broker email address
IMAP_PASS	Yes (for MCP)	Email account password

HOST	No	API server host. Default: 127.0.0.1
PORT	No	API server port. Default: 5000
SIGNAL_CACHE_REFRESH_MINUTES	No	Signal refresh interval. Default: 5
MATCH_TOP_N	No	Number of ranked results. Default: 5
WEIGHT_VESSEL_SIZE	No	Scoring weight. Default: 0.35
WEIGHT_GEOGRAPHY	No	Scoring weight. Default: 0.30
WEIGHT_DATE_OVERLAP	No	Scoring weight. Default: 0.20
WEIGHT_CARGO_TYPE	No	Scoring weight. Default: 0.15
PARSER_CONFIDENCE_THRESHOLD	No	Flag threshold. Default: 0.75

12.4 Local Ports

Port	Protocol	Used By	Notes
5000	HTTP + WebSocket	Python FastAPI backend	All backend traffic
5432	TCP	PostgreSQL	Local database only
3000	HTTP	React dev server	Development only — not in production

Note: All ports are localhost only. Nothing is exposed to the network. Outbound HTTPS to api.anthropic.com and apis.signalocean.com on port 443 must be permitted by any corporate firewall.

13 OPEN QUESTIONS & BLOCKERS

#	Question	Owner	Blocks
OQ-01	Does the broker's Signal Ocean subscription include Tonnage List, Distances, and Scraped Data APIs — or is separate API access required?	Panagiotis → Signal Ocean account manager	Signal MCP server build, live matching
OQ-02	What is the broker's mailbox host and does it support IMAP (port 993/143)?	Panagiotis → IT contact at broker firm	IMAP MCP configuration
OQ-03	Can IMAP credentials (host, port, username, password) be provided or a read-only mirror account created?	Panagiotis → IT contact at broker firm	IMAP MCP server build
OQ-04	Is IMAP access restricted to the office network — or accessible from development machines remotely?	Panagiotis → IT contact at broker firm	Development environment setup
OQ-05	What is the scope of the WT3 clone email client — full email management or read-only matching panel only?	Panagiotis + Developer	Frontend build estimate
OQ-06	Is there a preferred deployment environment — local machine only, a small server, or cloud?	Panagiotis	Infrastructure decisions

14 GLOSSARY

Term	Definition
MCP	Model Context Protocol. Anthropic's standard for giving Claude access to external tools and data sources.
IMAP	Internet Message Access Protocol. Standard protocol for reading emails from a mail server. Port 993 (TLS).
Laycan	Laydays/Cancelling. The window of dates within which a vessel must arrive at the load port.
DWT	Deadweight Tonnage. A measure of how much cargo a vessel can carry, in metric tons.
Aframax	Tanker vessel class. 80,000–120,000 DWT. Common for crude oil and fuel oil.
Suezmax	Tanker vessel class. 120,000–200,000 DWT. Named for maximum size to transit Suez Canal.
VLCC	Very Large Crude Carrier. Tanker vessel class. 200,000–320,000 DWT.
Panamax	Dry bulk vessel class. 60,000–80,000 DWT. Named for maximum size to transit Panama Canal.
Capesize	Dry bulk vessel class. 100,000+ DWT. Too large for Suez or Panama — routes around Cape.
Handymax	Dry bulk vessel class. 40,000–60,000 DWT. Common for grain, fertiliser, sugar.
ARA	Amsterdam-Rotterdam-Antwerp. Major European port range and trading hub.
CLAUDE.md	Project memory file read automatically by Claude Code at the start of every session.
Fixture	The act of confirming and agreeing a vessel charter — the binding commercial agreement.
Tonnage list	A list of vessels available for charter, showing open positions, dates, and commercial status.
Open position	A vessel that is available for charter — not currently fixed or on subs.
On subs	A vessel under negotiation — not yet fixed but subject to agreement.
Worldscale	A standardised tanker freight rate system. WS 65 means 65% of the flat rate for that route.